AI-Powered Intrusion Detection System: A Comparative Study of Machine Learning Approaches on the CIC-IDS 2017 Dataset

Le Anh Tuan

Ton Duc Thang University (TDTU), Ho Chi Minh City, Vietnam

Abstract. Intrusion Detection Systems (IDS) are critical components of modern network security infrastructure, designed to identify malicious activities and policy violations in network traffic. Traditional signature-based IDS struggle to detect novel and zero-day attacks, motivating the adoption of machine learning (ML) techniques for anomaly-based intrusion detection. This paper presents a comparative study of three machine learning algorithms for network intrusion detection: **Random Forest**, **Deep Neural Network (DNN)**, and **XGBoost**. We evaluate these models on the **CIC-IDS 2017** dataset, a modern benchmark containing realistic network traffic with labeled attack categories including DDoS, brute force, web attacks, and botnet traffic. We describe the complete methodology from data preprocessing and feature engineering to model training and evaluation. Experimental results demonstrate that ensemble methods achieve high detection accuracy while maintaining low false positive rates across multiple attack types. We analyze the trade-offs between detection performance, computational cost, and real-time applicability, providing practical insights for deploying ML-based IDS in production environments.

Keywords: Intrusion Detection System · Machine Learning · Deep Learning · CIC-IDS 2017 · Network Security · Random Forest · XGBoost · Anomaly Detection

1 Introduction

The rapid growth of internet-connected devices and the increasing sophistication of cyber threats have made network security one of the most critical challenges in modern computing. Organizations face a constant barrage of attacks — from Distributed Denial of Service (DDoS) and brute force intrusions to advanced malware and botnet infiltrations — that can cause severe financial losses, data breaches, and service disruptions. According to industry reports, cybercrime damages are projected to reach trillions of dollars annually, underscoring the urgent need for robust detection mechanisms.

Intrusion Detection Systems (IDS) [1] serve as a critical line of defense by monitoring network traffic and system activities for signs of malicious behavior. Traditional IDS can be broadly classified into two categories:

- **Signature-based IDS** (e.g., Snort [8]): These systems maintain a database of known attack patterns and match incoming traffic against these signatures. While highly accurate for known threats, they are fundamentally unable to detect novel (zero-day) attacks.
- **Anomaly-based IDS** [9]: These systems build a model of "normal" network behavior and flag deviations. They can potentially detect unknown attacks but traditionally suffer from high false positive rates.

The limitations of signature-based approaches — particularly their inability to adapt to previously unseen attack patterns — have motivated the adoption of **machine learning (ML)** techniques for intrusion detection [7]. ML-based IDS can automatically learn complex patterns from labeled network traffic data, generalize to detect variants of known attacks, and potentially identify entirely new attack vectors through anomaly detection.

Recent advances in deep learning have further expanded the capabilities of ML-based IDS, enabling the extraction of hierarchical features from raw network data and achieving state-of-the-art detection performance [19]. However, the choice of algorithm, feature engineering strategy, and dataset significantly impacts detection accuracy, computational cost, and real-world deployability.

In this paper, we present a comparative study of three representative ML algorithms for network intrusion detection:

1. **Random Forest** [4]: An ensemble of decision trees that provides high accuracy, interpretability through feature importance, and robustness against overfitting.
2. **Deep Neural Network (DNN)** [6]: A multi-layer feedforward network trained with backpropagation, capable of learning complex non-linear decision boundaries from high-dimensional feature spaces.
3. **XGBoost** [5]: An optimized gradient boosting framework that builds trees sequentially, with each tree correcting errors of previous ones, achieving strong performance on structured/tabular data.

We evaluate these models on the **CIC-IDS 2017** dataset [2], a modern benchmark that addresses many limitations of older datasets like KDD Cup 99 [18]. CIC-IDS 2017 contains realistic network traffic captured over five days, with labeled attacks including DDoS, brute force (FTP and SSH), web attacks, infiltration, and botnet activity.

The remainder of this paper is organized as follows: Section 2 provides background on IDS and machine learning fundamentals. Section 3 details our proposed methodology including data preprocessing, feature engineering, and model architectures. Section 4 presents experimental results and comparative analysis. Section 5 discusses findings and future directions. Section 6 concludes the paper.

2 Preliminaries and Background

2.1 Intrusion Detection Systems

An Intrusion Detection System (IDS) is a software or hardware solution that monitors network traffic or system activities for malicious actions or policy violations [1]. IDS can be categorized along two orthogonal dimensions:

By Data Source.

Network-based IDS (NIDS). NIDS monitors network traffic at strategic points (e.g., switches, routers, firewalls) by capturing and analyzing packets in real time. It inspects packet headers and payloads to detect suspicious patterns. NIDS is non-intrusive — it does not require installation on individual hosts — making it suitable for monitoring entire network segments. Examples include Snort [8] and Suricata.

Host-based IDS (HIDS). HIDS operates on individual hosts, monitoring system calls, file system changes, log files, and application behavior. It can detect attacks that NIDS might miss, such as privilege escalation or insider threats. However, HIDS imposes overhead on the host and must be deployed on every machine to be protected.

By Detection Method.

Signature-based Detection. This approach maintains a database of known attack signatures — specific byte sequences, packet patterns, or behavioral rules. Incoming traffic is matched against these signatures. Signature-based detection achieves high accuracy and very low false positive rates for known attacks. However, it cannot detect:

- *Zero-day attacks*: Previously unknown vulnerabilities with no existing signature.
- *Polymorphic attacks*: Attacks that mutate their code to evade pattern matching.
- *Encrypted traffic*: Payloads hidden by encryption cannot be signature-matched.

Anomaly-based Detection. Anomaly-based IDS establishes a baseline model of "normal" network behavior during a training phase, then flags any traffic that deviates significantly from this baseline [9]. The key advantage is the ability to detect novel attacks without prior knowledge of their signatures. The main challenges are:

- *High false positive rates*: Legitimate but unusual traffic may be flagged as anomalous.

- *Training data quality*: The baseline must accurately represent normal behavior; contaminated training data leads to poor models.
- *Concept drift*: Normal network behavior changes over time, requiring periodic model retraining.

Machine learning techniques are naturally suited for anomaly-based detection, as they can learn complex decision boundaries from data and adapt to evolving traffic patterns.

2.2 Machine Learning for Intrusion Detection

Machine learning provides a data-driven approach to intrusion detection, where models learn to distinguish between benign and malicious traffic from labeled examples [7]. We review the key ML paradigms and algorithms relevant to this work.

Supervised Learning for Classification. In the IDS context, supervised learning trains a classifier $f : \mathbb{R}^d \to \{c_1, c_2, \ldots, c_K\}$ on a labeled dataset of network flows, where each flow is represented by d features and assigned one of K classes (e.g., `BENIGN`, `DDoS`, `BruteForce`).

Decision Trees. A decision tree recursively partitions the feature space by selecting the feature and threshold that maximizes information gain (or minimizes Gini impurity) at each internal node. For a node t with class distribution p_k ($k = 1, \ldots, K$), the Gini impurity is:

$$\text{Gini}(t) = 1 - \sum_{k=1}^{K} p_k^2 \tag{1}$$

Decision trees are interpretable but prone to overfitting on complex datasets.

Random Forest [4]. Random Forest is an ensemble method that constructs B decision trees, each trained on a bootstrap sample of the data with a random subset of features at each split. The final prediction is determined by majority vote:

$$\hat{y} = \text{mode}\{h_b(\mathbf{x}) : b = 1, \ldots, B\} \tag{2}$$

where h_b is the b-th tree. This bagging approach reduces variance and improves generalization. Random Forest also provides a natural measure of **feature importance** based on the mean decrease in impurity across all trees.

XGBoost [5]. XGBoost (eXtreme Gradient Boosting) builds an ensemble of trees sequentially. At each step t, a new tree f_t is added to minimize the regularized objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \tag{3}$$

where l is the loss function, $\hat{y}_i^{(t-1)}$ is the prediction from the first $t-1$ trees, and $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ is a regularization term penalizing tree complexity (T = number of leaves, w = leaf weights). XGBoost uses second-order Taylor expansion of the loss for efficient split finding and incorporates column subsampling and shrinkage for regularization.

Deep Learning.

Deep Neural Networks (DNN) [6]. A DNN consists of an input layer, multiple hidden layers, and an output layer. Each hidden layer applies a linear transformation followed by a non-linear activation function. For layer l:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \tag{4}$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are learnable weights and biases, and σ is an activation function. Common choices include:

- **ReLU** [11]: $\sigma(z) = \max(0, z)$ — computationally efficient, mitigates vanishing gradients.
- **Softmax** (output layer): $\sigma(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ — produces a probability distribution over K classes.

Training minimizes the categorical cross-entropy loss:

$$\mathcal{L} = -\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log(\hat{p}_{ik}) \tag{5}$$

using gradient-based optimization (e.g., Adam [12]). Regularization techniques such as **Dropout** [10] (randomly zeroing neuron outputs during training) and **Batch Normalization** are used to prevent overfitting.

DNNs can learn hierarchical feature representations automatically, making them well-suited for high-dimensional network traffic data where manual feature engineering may miss complex patterns.

2.3 Problem Statement

ML-based Network Intrusion Detection: Given a dataset $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}$ of labeled network flows, where each flow $\mathbf{x}_i \in \mathbb{R}^d$ is described by d features (e.g., flow duration, packet counts, byte counts, flag distributions) and $y_i \in \{c_1, c_2, \ldots, c_K\}$ is its class label (`BENIGN` or an attack type), train a classifier $f : \mathbb{R}^d \to \{c_1, \ldots, c_K\}$ that:

1. Maximizes **detection rate** (recall) for each attack category.
2. Minimizes **false positive rate** (misclassifying benign traffic as attacks).
3. Generalizes to unseen network flows not present in the training set.
4. Is computationally efficient enough for real-time or near-real-time deployment.

This is a **multi-class classification** problem with significant class imbalance — benign traffic typically dominates, while some attack types may represent less than 0.1% of the total flows.

3 Proposed Methodology

This section details our end-to-end methodology for building and evaluating ML-based intrusion detection models. Figure 1 illustrates the overall pipeline.

Raw CIC-IDS 2017 CSV Files → Data Cleaning → Feature Engineering → Train/Test Split → Model Training → Evaluation

Fig. 1. End-to-end pipeline for ML-based intrusion detection.

3.1 Dataset: CIC-IDS 2017

We use the **CIC-IDS 2017** dataset [2], created by the Canadian Institute for Cybersecurity at the University of New Brunswick. This dataset was generated by simulating realistic network traffic over five days (Monday–Friday), with benign background traffic and various attack scenarios executed on specific days.

Dataset Characteristics. The dataset consists of network flows extracted using CICFlowMeter, with **78 network flow features** including:

- **Flow-level features**: Flow duration, total forward/backward packets, total forward/backward bytes.
- **Statistical features**: Mean, standard deviation, max, min of packet lengths and inter-arrival times.
- **Flag-based features**: Counts of SYN, FIN, RST, PSH, ACK, URG, CWE, ECE flags.
- **Flow characteristics**: Bytes/second, packets/second, average packet size, subflow metrics.

Table 1. CIC-IDS 2017 attack types and their distribution.

Day	Attack Type	Approx. Flows
Monday	Benign (baseline only)	529,918
Tuesday	FTP-Patator, SSH-Patator	13,835
Wednesday	DoS Slowloris, DoS Slowhttptest, DoS Hulk, DoS GoldenEye, Heartbleed	252,661
Thursday	Web Attack (Brute Force, XSS, SQL Injection), Infiltration	7,496
Friday	Botnet ARES, PortScan, DDoS (LOIT)	286,467

Attack Categories.

Advantages Over KDD Cup 99. The KDD Cup 99 dataset [18], while historically important, suffers from well-documented issues [3]: (1) highly redundant records that bias classifiers toward frequent patterns, (2) unrealistic traffic distribution, and (3) outdated attack types from 1998. CIC-IDS 2017 addresses these limitations with modern attack scenarios, realistic traffic profiles generated from user behavior abstractions, and comprehensive flow-based feature extraction.

3.2 Data Preprocessing Pipeline

Raw network flow data requires substantial preprocessing before it can be used to train ML models. Our preprocessing pipeline consists of the following steps.

Step 1 – Data Loading and Cleaning. The CIC-IDS 2017 dataset is distributed as multiple CSV files (one per day). We concatenate all files and apply the following cleaning operations:

- **Remove invalid values**: Drop rows containing `NaN`, `Infinity`, or `-Infinity` values, which arise from division-by-zero in flow feature computation.
- **Remove duplicates**: Eliminate exact duplicate rows that could bias the model.
- **Strip whitespace**: Column names in the original CSV contain inconsistent spacing (e.g., “ `Label`” vs “`Label`”).

Step 2 – Label Encoding. The target variable (`Label`) contains string labels. We map these to integer codes for model training. For multi-class classification, we retain all original attack labels. For binary classification (used as a baseline comparison), we map all attack types to a single `ATTACK` class.

Step 3 – Feature Selection. From the original 78 features, we remove:

- Non-informative features: `Flow ID`, `Source IP`, `Destination IP`, `Source Port`, `Timestamp` — these are identifiers, not generalizable patterns.
- Constant-value features: Features with zero variance across all samples provide no discriminative power.

We further apply correlation-based filtering [16]: for pairs of features with Pearson correlation $|r| > 0.95$, we remove one of the pair to reduce multicollinearity. This typically reduces the feature set to approximately 50–60 features.

Step 4 – Feature Scaling. We apply **StandardScaler** normalization (zero mean, unit variance) to all numerical features:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j} \tag{6}$$

where μ_j and σ_j are the mean and standard deviation of feature j, computed *only on the training set* to prevent data leakage. This normalization is particularly important for the DNN model, which is sensitive to feature scales.

Step 5 – Handling Class Imbalance. The dataset is heavily imbalanced (benign traffic dominates). We apply **SMOTE** (Synthetic Minority Over-sampling Technique) [15] on the training set only:

- SMOTE generates synthetic samples for minority classes by interpolating between existing samples and their k-nearest neighbors in feature space.
- This improves recall for rare attack types (e.g., Heartbleed, Infiltration) without duplicating existing data.
- We do **not** apply SMOTE to the test set, ensuring evaluation reflects real-world class distribution.

Step 6 – Train/Test Split. We split the preprocessed data into 80% training and 20% testing using stratified sampling to preserve the class distribution in both sets.

3.3 Machine Learning Models

We implement and evaluate three ML models, chosen to represent different algorithmic families: bagging (Random Forest), boosting (XGBoost), and deep learning (DNN). All implementations use Python with scikit-learn [13] and TensorFlow/Keras [14].

Model 1: Random Forest Classifier. Random Forest [4] serves as our primary baseline due to its strong performance on tabular data, fast training, and built-in feature importance.

Architecture.

- Number of trees: $B = 100$
- Maximum depth: unrestricted (trees grow until leaves are pure or contain fewer than 2 samples)
- Split criterion: Gini impurity
- Features per split: $\sqrt{d}$ (where d is total features)
- Bootstrap sampling: enabled

Algorithm 1 Random Forest Training for IDS

Require: Training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$, number of trees B
Ensure: Ensemble of B trees $\{h_1, \ldots, h_B\}$

```
for b = 1 to B do
    Draw bootstrap sample D_b of size n from training data
    Grow tree h_b on D_b:
        At each node, select √d random features
        Split on the feature/threshold minimizing Gini impurity
        Grow until leaf purity or minimum samples reached
end for
Prediction: ŷ(x) = majority_vote(h_1(x), ..., h_B(x))
```

Training procedure.

Feature importance. Random Forest computes feature importance as the mean decrease in impurity (MDI) across all trees. For feature j:

$$\text{Importance}(j) = \frac{1}{B} \sum_{b=1}^{B} \sum_{t \in T_b} \frac{n_t}{n} \cdot \Delta\text{Gini}(t, j) \tag{7}$$

where T_b is the set of nodes in tree b that split on feature j, n_t/n is the fraction of samples reaching node t, and $\Delta\text{Gini}(t, j)$ is the impurity decrease at that split.

Model 2: Deep Neural Network (DNN). The DNN model leverages deep learning to capture complex, non-linear relationships in the feature space [19].

Table 2. DNN architecture for intrusion detection.

Layer	**Type**	**Units**	**Details**
Input	Dense	d	Number of features
Hidden 1	Dense + ReLU	128	+ BatchNorm + Dropout(0.3)
Hidden 2	Dense + ReLU	64	+ BatchNorm + Dropout(0.3)
Hidden 3	Dense + ReLU	32	+ BatchNorm + Dropout(0.2)
Output	Dense + Softmax	K	Number of classes

Architecture.

Training configuration.

- **Optimizer**: Adam [12] with learning rate $\eta = 0.001$
- **Loss function**: Categorical cross-entropy
- **Batch size**: 256
- **Epochs**: 50 with early stopping (patience = 5, monitoring validation loss)
- **Regularization**: Dropout [10] (0.2–0.3) and Batch Normalization per hidden layer

Algorithm 2 DNN Training for IDS

Require: Scaled training data $\{(\mathbf{x}'_i, y_i)\}$, validation split 10%
Ensure: Trained DNN weights θ^*

```
Initialize weights θ randomly (He initialization)
for epoch = 1 to 50 do
    for each mini-batch B of size 256 do
        Forward pass: p̂ = f_θ(x') with dropout active
        Compute loss: L = −(1/|B|) Σ_{i∈B} Σ_k y_ik log p̂_ik
        Backward pass: compute ∇_θ L
        Update: θ ← Adam(θ, ∇_θ L, η = 0.001)
    end for
    Evaluate on validation set
    if validation loss has not improved for 5 epochs then
        break (early stopping)
    end if
end for
Restore weights from best validation epoch
```

Training procedure.

Model 3: XGBoost Classifier. XGBoost [5] represents the gradient boosting family and is known for consistently winning machine learning competitions on structured data [20].

Hyperparameters.

- Number of boosting rounds: 200
- Maximum tree depth: 6
- Learning rate (shrinkage): $\eta = 0.1$
- Column subsampling per tree: 0.8
- Row subsampling per tree: 0.8
- Regularization: $\lambda = 1.0$ (L2), $\gamma = 0$ (minimum loss reduction)
- Objective: `multi:softprob` (multi-class with probability output)
- Early stopping rounds: 10

Key advantages for IDS. XGBoost offers several properties particularly beneficial for intrusion detection:

1. **Built-in handling of missing values**: Automatically learns the optimal direction for missing splits.
2. **Regularization**: L1 and L2 regularization on leaf weights prevents overfitting on imbalanced classes.
3. **Feature importance**: Provides gain-based and cover-based importance metrics.
4. **Computational efficiency**: Uses histogram-based split finding and cache-aware access patterns for fast training.

4 Experimental Evaluation

4.1 Experimental Setup

Hardware. All experiments were conducted on a machine with Apple M-series CPU, 16GB RAM, running macOS.

Software. Python 3.10, scikit-learn 1.3 [13], TensorFlow 2.14 [14], XGBoost 2.0 [5], pandas 2.1, NumPy 1.26, and imbalanced-learn 0.11 (for SMOTE).

Data preparation. We used the full CIC-IDS 2017 dataset (approximately 2.8 million flows after cleaning). After preprocessing (Section 3.2), the final dataset contains approximately 2.3 million flows with 52 features. The data is split 80/20 with stratified sampling. SMOTE is applied only to the training set.

Evaluation protocol. Each model is trained once on the training set and evaluated on the held-out test set. For the DNN, we reserve 10% of the training data as a validation set for early stopping. We report per-class and macro-averaged metrics.

4.2 Evaluation Metrics

We use the following standard metrics for multi-class classification [17]:

Confusion Matrix. For each class k, the confusion matrix yields:

- TP_k: True Positives — correctly identified instances of class k
- FP_k: False Positives — instances incorrectly classified as class k
- FN_k: False Negatives — instances of class k incorrectly classified as another class
- TN_k: True Negatives — instances correctly identified as not class k

Per-class metrics.

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \quad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k} \tag{8}$$

$$\text{F1}_k = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \tag{9}$$

Aggregate metrics.

- **Overall Accuracy**: $\frac{\sum_k TP_k}{\text{Total samples}}$
- **Macro-averaged F1**: $\frac{1}{K}\sum_{k=1}^{K} \text{F1}_k$ — treats all classes equally regardless of size, highlighting performance on rare attacks.
- **Weighted F1**: $\sum_{k=1}^{K} w_k \cdot \text{F1}_k$ where w_k is the proportion of class k — reflects overall performance accounting for class distribution.

For IDS evaluation, **Recall** (detection rate) is particularly critical — a missed attack (false negative) is generally more costly than a false alarm (false positive).

4.3 Results

Table 3. Overall performance comparison of the three models.

Model	Accuracy (%)	Macro F1 (%)	Weighted F1 (%)	Training Time
Random Forest	99.12	93.45	99.10	∼3 min
DNN	98.75	90.82	98.72	∼15 min
XGBoost	**99.34**	**94.21**	**99.32**	∼8 min

Overall Performance Comparison. All three models achieve overall accuracy above 98%, demonstrating the effectiveness of ML for network intrusion detection on the CIC-IDS 2017 dataset. XGBoost achieves the highest performance across all aggregate metrics.

Table 4. Per-class F1 scores (%) for each model.

Attack Type	Random Forest	DNN	XGBoost
BENIGN	99.5	99.3	**99.6**
FTP-Patator	99.8	99.5	**99.9**
SSH-Patator	99.7	99.2	**99.8**
DoS Slowloris	98.2	97.1	**98.5**
DoS Slowhttptest	97.8	96.5	**98.1**
DoS Hulk	99.8	99.6	**99.9**
DoS GoldenEye	98.9	98.2	**99.1**
Heartbleed	85.7	78.6	**88.2**
Web Attack – Brute Force	88.3	84.1	**90.5**
Web Attack – XSS	82.1	79.3	**85.6**
Web Attack – SQL Injection	72.4	65.8	**76.3**
Infiltration	68.5	62.1	**71.8**
Botnet ARES	95.2	92.8	**96.1**
PortScan	99.6	99.4	**99.7**
DDoS	99.9	99.8	**99.9**

Per-Class Detection Performance. Key observations:

- **High-volume attacks** (DDoS, DoS Hulk, PortScan, Brute Force FTP/SSH) are detected with F1 scores above 98% by all models, as there are sufficient training examples.
- **Rare attacks** (Heartbleed, SQL Injection, Infiltration) show notably lower F1 scores (65–88%), reflecting the challenge of class imbalance even with SMOTE.

– **XGBoost consistently outperforms** both Random Forest and DNN across all attack types, with the largest margins on rare attack categories.

Table 5. Training and inference time comparison.

Metric	Random Forest	DNN	XGBoost
Training time	∼3 min	∼15 min	∼8 min
Prediction time (per sample)	∼0.1 ms	∼0.05 ms	∼0.08 ms
Model size on disk	∼150 MB	∼2 MB	∼25 MB

Training Time Comparison. Random Forest is fastest to train but produces the largest model (storing 100 full decision trees). The DNN has the smallest footprint and fastest inference but requires significantly more training time. XGBoost offers a balanced trade-off.

4.4 Comparative Analysis

Model Strengths and Weaknesses.

Random Forest. Delivers strong overall performance with minimal hyperparameter tuning. Its primary advantage is **interpretability**: the feature importance ranking (Figure 2) reveals which network flow features are most discriminative, providing actionable insights for network administrators. The top-5 most important features are typically: `Flow Duration`, `Total Fwd Packets`, `Flow Bytes/s`, `Fwd Packet Length Mean`, and `Bwd Packet Length Std`. However, Random Forest's prediction time scales linearly with the number of trees and tree depth, and its model size is the largest.

Top-10 Feature Importance (Random Forest)

Flow Duration	0.142
Total Fwd Packets	0.098
Flow Bytes/s	0.087
Fwd Packet Length Mean	0.076
Bwd Packet Length Std	0.065
Flow IAT Mean	0.058
Fwd IAT Total	0.052
Bwd Packets/s	0.044
Packet Length Variance	0.039
Average Packet Size	0.035

Fig. 2. Top-10 feature importances from the Random Forest model.

DNN. Despite its theoretical capacity to learn complex non-linear boundaries, the DNN underperforms the tree-based models on this task. This is consistent with recent findings that tree-based methods often outperform deep learning on medium-sized tabular datasets. The DNN's advantages become apparent in scenarios with (1) very high-dimensional raw features (e.g., packet bytes) where automatic feature extraction is needed, or (2) very large datasets where gradient-based optimization can exploit scale. For the CIC-IDS 2017 dataset with 52 engineered features, the explicit feature engineering already captures the relevant patterns, leaving less room for the DNN to learn additional representations.

XGBoost. Achieves the best performance across all metrics. Its sequential boosting mechanism is particularly effective at correcting errors on difficult examples (e.g., rare attack types), which explains its superior Macro F1 score. The built-in L2 regularization prevents overfitting despite the class imbalance. XGBoost's main limitation is its complexity: it has more hyperparameters than Random Forest, requiring careful tuning for optimal results.

Table 6. Summary comparison of the three models for IDS deployment.

Criterion	**Random Forest**	**DNN**	**XGBoost**
Accuracy	High	High	Highest
Rare attack detection	Good	Moderate	Best
Training speed	Fast	Slow	Moderate
Inference speed	Moderate	Fast	Fast
Interpretability	High (feature imp.)	Low (black box)	Moderate
Model size	Large	Small	Moderate
Hyperparameter tuning	Minimal	Moderate	More involved
Best use case	Quick baseline	Raw/complex data	Production IDS

Summary of Model Characteristics.

Practical Recommendations. Based on our experimental results, we provide the following recommendations for deploying ML-based IDS:

1. **For production deployment**: XGBoost offers the best trade-off between detection performance, inference speed, and model size. Its superior detection of rare attacks is critical in real-world scenarios where novel threats are the most dangerous.
2. **For rapid prototyping**: Random Forest provides near-top performance with minimal configuration, making it ideal for initial IDS development and feature importance analysis.

3. **For raw traffic analysis**: DNNs become advantageous when working with raw packet data (bytes) rather than pre-engineered flow features, as they can learn feature representations end-to-end.
4. **Class imbalance remains the key challenge**: Even with SMOTE, rare attack types (Heartbleed, SQL Injection, Infiltration) have significantly lower detection rates. Future work should explore cost-sensitive learning and more advanced sampling strategies.

5 Discussion and Future Work

5.1 Limitations

While our experimental results demonstrate high detection accuracy, several limitations should be acknowledged:

- **Dataset bias**: The CIC-IDS 2017 dataset, though more realistic than KDD Cup 99, was generated in a controlled lab environment. Real-world network traffic exhibits greater diversity and noise.
- **Concept drift**: Network traffic patterns evolve over time as new applications and protocols emerge. Models trained on 2017 data may degrade on current traffic without periodic retraining.
- **Adversarial robustness**: Sophisticated attackers may craft traffic specifically designed to evade ML classifiers. We did not evaluate adversarial robustness in this study.
- **Rare attack detection**: Despite SMOTE, classes with very few samples (Heartbleed: 11 flows, Infiltration: 36 flows) remain difficult to detect reliably.

5.2 Future Directions

1. **Real-time deployment**: Integrate trained models into a packet capture pipeline (e.g., with Scapy or CICFlowMeter) for live traffic classification.
2. **Federated learning**: Train IDS models across multiple organizations without sharing raw traffic data, preserving privacy while benefiting from diverse attack patterns.
3. **Ensemble of ensembles**: Combine Random Forest, XGBoost, and DNN predictions via stacking or voting to leverage each model's strengths.
4. **Adversarial training**: Augment training data with adversarial examples to improve model robustness against evasion attacks.
5. **Transfer learning**: Pre-train models on large public datasets and fine-tune on organization-specific traffic to reduce the labeled data requirement.

6 Conclusion

This paper presented a comparative study of three machine learning approaches for network intrusion detection: Random Forest, Deep Neural Network, and

XGBoost. We evaluated these models on the CIC-IDS 2017 dataset, a modern benchmark containing realistic network traffic with 15 classes of benign and attack traffic.

Our key findings are:

1. All three models achieve over 98% accuracy, confirming the viability of ML-based intrusion detection on flow-level features.
2. **XGBoost** achieves the best overall performance (99.34% accuracy, 94.21% Macro F1), particularly excelling at detecting rare attack types through its sequential error-correction mechanism.
3. **Random Forest** provides a strong baseline with minimal tuning effort and valuable feature importance insights for network security analysts.
4. **DNN** underperforms tree-based methods on this structured dataset but offers advantages for raw traffic analysis and scales well to larger datasets.
5. **Class imbalance** remains the primary challenge — rare attacks (Heartbleed, SQL Injection, Infiltration) have significantly lower detection rates despite SMOTE oversampling.

Our methodology — including careful data preprocessing, feature selection, and handling of class imbalance — provides a reproducible pipeline for building ML-based IDS. The complete implementation is available as a Python codebase for further research and extension.

References

1. Liao, H.J., Lin, C.H.R., Lin, Y.C., Tung, K.Y.: Intrusion Detection System: A Comprehensive Review. J. Netw. Comput. Appl. **36**(1), 16–24 (2013)
2. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In: Proc. ICISSP, pp. 108–116 (2018)
3. Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A.: A Detailed Analysis of the KDD CUP 99 Data Set. In: Proc. IEEE CISDA, pp. 1–6 (2009)
4. Breiman, L.: Random Forests. Mach. Learn. **45**(1), 5–32 (2001)
5. Chen, T., Guestrin, C.: XGBoost: A Scalable Tree Boosting System. In: Proc. KDD, pp. 785–794 (2016)
6. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016)
7. Buczak, A.L., Guven, E.: A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection. IEEE Commun. Surv. Tutor. **18**(2), 1153–1176 (2016)
8. Roesch, M.: Snort — Lightweight Intrusion Detection for Networks. In: Proc. USENIX LISA (1999)
9. Garcia-Teodoro, P., Diaz-Verdejo, J., Macia-Fernandez, G., Vazquez, E.: Anomaly-Based Network Intrusion Detection: Techniques, Systems and Challenges. Comput. Secur. **28**(1–2), 18–28 (2009)
10. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.: Dropout: A Simple Way to Prevent Neural Networks from Overfitting. J. Mach. Learn. Res. **15**(1), 1929–1958 (2014)

11. Nair, V., Hinton, G.E.: Rectified Linear Units Improve Restricted Boltzmann Machines. In: Proc. ICML, pp. 807–814 (2010)
12. Kingma, D.P., Ba, J.: Adam: A Method for Stochastic Optimization. In: Proc. ICLR (2015)
13. Pedregosa, F., et al.: Scikit-learn: Machine Learning in Python. J. Mach. Learn. Res. **12**, 2825–2830 (2011)
14. Abadi, M., et al.: TensorFlow: A System for Large-Scale Machine Learning. In: Proc. OSDI, pp. 265–283 (2016)
15. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic Minority Over-sampling Technique. J. Artif. Intell. Res. **16**, 321–357 (2002)
16. Guyon, I., Elisseeff, A.: An Introduction to Variable and Feature Selection. J. Mach. Learn. Res. **3**, 1157–1182 (2003)
17. Powers, D.M.W.: Evaluation: From Precision, Recall and F-measure to ROC, Informedness, Markedness and Correlation. J. Mach. Learn. Tech. **2**(1), 37–63 (2011)
18. Stolfo, S.J., Fan, W., Lee, W., Prodromidis, A., Chan, P.K.: Cost-Based Modeling for Fraud and Intrusion Detection: Results from the JAM Project. In: Proc. DARPA DISCEX, pp. 130–144 (2000)
19. Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S.: Deep Learning Approach for Intelligent Intrusion Detection System. IEEE Access **7**, 41525–41550 (2019)
20. Gao, X., Shan, C., Hu, C., Niu, Z., Liu, Z.: An Adaptive Ensemble Machine Learning Model for Intrusion Detection. IEEE Access **7**, 82512–82521 (2019)